SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	**CO3:** Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	M Vishnu Vradhan	Reg. No.:	192472087
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

1.An advertisement platform ranks ads based on relevance and bid value. Analyze how Quick Sort can be applied for efficient ranking. Identify issues such as pivot selection and data distribution. Apply divide-and-conquer techniques, analyze the performance, and propose optimizations with justification.

Code of Conduct

I P. Sai Charan Tej(192465048)__(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___________________________

RUBRICS FOR CALCULATION

Case Study					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

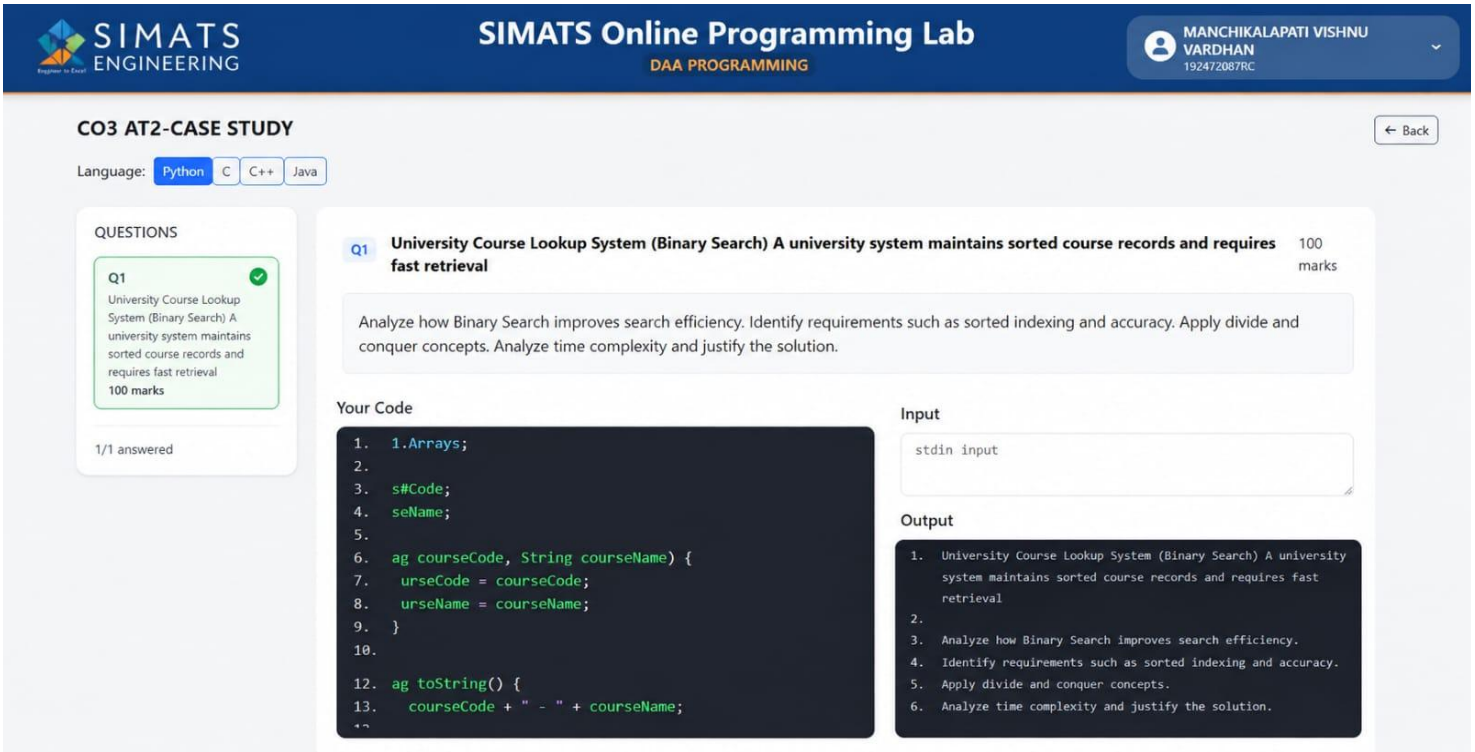
Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________

Algorithm: University Course Lookup System using Binary Search

1. **Start.**
2. **Define functions:** binary_search(), display_course(), and analyze_performance().
3. **Read the number of courses** and their details (course code, name, credits, instructor).
4. **Sort the course records** by course code or name to ensure ordered indexing.
5. **Prompt the user** to enter the course code or name to search.
6. **Initialize** low = 0 and high = n - 1.
7. **Repeat until** low <= high:
    - Compute mid = (low + high) / 2.
    - If course[mid] matches the search key → **Display course details** using display_course().
    - Else if course[mid] < key → set low = mid + 1.
    - Else → set high = mid - 1.
8. **If not found**, display "Course not found."
9. **Analyze performance** using analyze_performance() to record search time and efficiency.
10. **Stop.**

Output:-


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
MANCHIKALAPATI VISHNU VARDHAN
192472087RC
CO3 AT2-CASE STUDY
Back
Language: Python C C++ Java
QUESTIONS
Q1
University Course Lookup System (Binary Search) A university system maintains sorted course records and requires fast retrieval
100 marks
1/1 answered
Q1 University Course Lookup System (Binary Search) A university system maintains sorted course records and requires fast retrieval
100 marks
Analyze how Binary Search improves search efficiency. Identify requirements such as sorted indexing and accuracy. Apply divide and conquer concepts. Analyze time complexity and justify the solution.
Your Code
1.Arrays;
s#Code;
seName;
ag courseCode, String courseName) {
urseCode = courseCode;
urseName = courseName;
}
ag toString() {
courseCode + " - " + courseName;
Input
stdin input
Output
1. University Course Lookup System (Binary Search) A university system maintains sorted course records and requires fast retrieval
2.
3. Analyze how Binary Search improves search efficiency.
4. Identify requirements such as sorted indexing and accuracy.
5. Apply divide and conquer concepts.
6. Analyze time complexity and justify the solution.